
Computer lab exercises 4

TANA21/22: BERÄKNINGSMATEMATIK

Andrew R. Winters

HT1 2026

Name: _____ Group: _____

LiU ID: _____

E-mail: _____

Name: _____ Group: _____

LiU ID: _____

E-mail: _____

Approved: _____ Signature: _____

Corrections: _____

The goal of this set of computer exercises is to explore:

1. Numerical integration (quadrature), their associated errors, and the *degree of exactness*
2. Composite quadrature and its associated error
3. Adaptive quadrature and a posteriori error analysis
4. Solving ordinary differential equations, specifically initial value problems, with a computer

4.1 Simpson's quadrature rule

Consider the definite integral

$$I(f) = \int_a^b f(x) \, dx,$$

for an arbitrary integrand function $f(x)$. Simpson's rule to approximate a definite integral is defined by the formula

$$\mathcal{S}_f(a, b) = \frac{h}{3} \left(f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right), \quad h = \frac{b-a}{2},$$

where we use the notation $\mathcal{S}_f(a, b)$ for the Simpson's rule on a given interval $[a, b]$. This is done because it helps to keep track of the interval to which the quadrature rule is applied. This notation is helpful later in the lab to estimate errors in Exercise 3.

- (a) Use pen-and-paper to compute the exact value of the definite integral

$$I(f) = \int_a^b 8x^3 - 4x^2 + 6x - 3 \, dx.$$

This will allow you to create a reference solution to check the accuracy of Simpson's rule for two bounds of integration:

- $a = 0, b = 1$

- $a = -4, b = 2$

- (b) Download the skeleton program `SimpsonsRule.py` and implement the formula for Simpson's rule given above. The input arguments are:

- The integrand function $f(x)$ that is treated as an anonymous function.
- The interval bounds a and b .
- The number of intervals n .

For now, set $n = 2$. You will expand the functionality of this function to take more intervals in the next Exercise. The output variables are

- The approximate value of the integral, saved in the variable `S`.
 - The $n + 1$ vector containing the quadrature nodes `x`.
- (c) The degree of exactness for Simpson's rule is $m = 3$ (see Exercise 6.4 in Exercise set 6). Therefore, Simpson's rule can integrate cubic polynomials exactly. Use this fact to verify your implementation of Simpson's rule by applying it to the integral from Part (a) and compute the error between the exact value and the numerical approximation.

`abs(I - S(0,1)) =`

`abs(I - S(-4,2)) =`

4.2 Composite Simpson's quadrature rule

Simpson's rule is quick and accurate, but a composite quadrature rule is even better for large intervals or more complicated functions. The composite form of Simpson's rule has the form

$$\mathcal{S}_f^{\text{comp}}(a, b) = \frac{h}{3} (f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + \cdots + 2f(x_{n-2}) + 4f(x_{n-1}) + f(x_n)),$$

provided the number of intervals n is **even**. The quadrature nodes $\{x_i\}_{i=0}^n$ are uniformly spaced with the form

$$h = \frac{b-a}{n}, \quad x_i = a + ih, \quad i = 0, \dots, n, \quad \text{such that} \quad x_0 = a, \quad x_n = b.$$

Note, if $n = 2$ this formula becomes the "classic" Simpson's rule from Exercise 1.

- (a) Change your implementation from Exercise 1 to be the composite Simpson's rule. Special attention must be paid to:

- The scaling constant on the function values between $i = 1, \dots, n-1$ changes depending on whether the index is odd or even. One way you can check is to use modular arithmetic. For example, the command

`i % 2 == 0`

would indicate the value stored in the variable `i` is an even number.

- Make sure the correct value of h is used in the formula.

- (b) The error term for the composite Simpson's rule on n intervals is

$$E_n^{\text{comp}} = -\frac{(b-a)}{180} h^4 f^{(4)}(\xi), \quad \xi \in [a, b].$$

Therefore, if we double the number of intervals to be $2n$ the error term becomes

$$E_{2n}^{\text{comp}} \simeq -\frac{(b-a)}{180} \left(\frac{h}{2}\right)^4 f^{(4)}(\xi) = -\frac{1}{2^4} \frac{(b-a)}{180} h^4 f^{(4)}(\xi) = \frac{E_n^{\text{comp}}}{16}, \quad \xi \in [a, b],$$

where we assume that the value of ξ is the same for each of the composite rules. This is valid as long as the function $f^{(4)}$ is “well-behaved” on the interval $[a, b]$. So, ideally, the ratio of these two errors is

$$\frac{E_n^{\text{comp}}}{E_{2n}^{\text{comp}}} = \frac{E_n^{\text{comp}}}{\frac{E_n^{\text{comp}}}{16}} = 16.$$

Use this error analysis to validate your implementation of composite Simpson’s rule with the integral

$$I = \int_1^2 \sqrt{x} \, dx = \frac{2}{3}(2\sqrt{2} - 1),$$

and fill in the rest of the table below.

n	$ E_n^{\text{comp}} = I - \mathcal{S}_f^{\text{comp}} $	Ratio
2	8.5909e-05	—
4	6.2596e-06	13.72
8		
16		

Does the error ratio match the expected value?

(c) Investigate the error ratio for the definite integral

$$I = \int_0^1 \sqrt{x} \, dx = \frac{2}{3},$$

and fill in the rest of the table below.

n	$ E_n^{\text{comp}} = I - \mathcal{S}_f^{\text{comp}} $	Ratio
2	2.8595e-02	—
4	1.0140e-02	2.82
8		
16		

On this new interval of integration does the error ratio match the expected value? Do our assumptions from the error analysis at the top of the page still hold?

4.3 Adaptive Simpson's quadrature rule

Next, we examine the application of Simpson's quadrature rule to a definite integral that is a bit more complicated. To do so, consider the integrand

$$f(x) = (x + 1)^2 \cos\left(\frac{2x + 1}{x - 3.3}\right),$$

for the definite integral

$$\int_0^3 f(x) dx = -1.564959809251273.$$

We can create the integrand function and the reference value for the integral with the commands

```
f = lambda x: (x + 1)**2 * np.cos((2 * x + 1) / (x - 3.3))
I_true = -1.564959809251273
```

- (a) Run your composite Simpson's rule for this integrand $f(x)$ on the domain $[0, 3]$ for $n = 10$ and $n = 20$ and report the result:

I10 =

I20 =

Does doubling the number of intervals improve the approximation of the integral?

- (b) Download the script `plot_composite_quad.py` from the course documents and use it to plot the quadrature points against the integrand to see what is happening. If necessary rerun your composite Simpson's rule on $n = 20$ intervals and be sure to return the set of quadrature points $\{x_i\}_{i=0}^n$

```
I20, x20 = SimpsonsRule(f, 0, 3, 20)
```

Does this uniform distribution of quadrature nodes capture the integrand? If not, explain why?

For this integrand nonuniformly distributed nodes would make more sense. In what follows we will examine how to select such nodes in an automatic way. Recall the error estimate for composite Simpson's rule in the previous Exercise required knowledge of the fourth derivative $f^{(4)}$. For the current integrand

$$f(x) = (x + 1)^2 \cos\left(\frac{2x + 1}{x - 3.3}\right),$$

finding this fourth derivative and analysing it is a **huge** pain. But we can estimate the term $f^{(4)}$ from the error using our knowledge from Exercise 2. Suppose the composite Simpson's rule is applied on two intervals

($n = 2$) and four intervals ($n = 4$). We then have the errors

$$\begin{aligned} E_2^{\text{comp}} &= I(f) - \mathcal{S}_f(a, b) &= -\frac{(b-a)}{180} h^4 f^{(4)}(\xi), \quad \xi \in [a, b], \\ E_4^{\text{comp}} &= I(f) - \mathcal{S}_f\left(a, \frac{a+b}{2}\right) - \mathcal{S}_f\left(\frac{a+b}{2}, b\right) &\simeq -\frac{1}{16} \frac{(b-a)}{180} h^4 f^{(4)}(\xi), \quad \xi \in [a, b]. \end{aligned}$$

Subtracting these two expressions and manipulating it is possible to estimate this fourth derivative term

$$\frac{16}{15} \left(\mathcal{S}_f(a, b) - \mathcal{S}_f\left(a, \frac{a+b}{2}\right) - \mathcal{S}_f\left(\frac{a+b}{2}, b\right) \right) \approx \frac{(b-a)}{180} h^4 f^{(4)}(\xi).$$

So the error on four intervals ($n = 4$) can be approximated **without** any knowledge of the function $f^{(4)}$

$$|E_4^{\text{comp}}| = \left| I(f) - \mathcal{S}_f\left(a, \frac{a+b}{2}\right) - \mathcal{S}_f\left(\frac{a+b}{2}, b\right) \right| \approx \frac{1}{15} \left| \mathcal{S}_f(a, b) - \mathcal{S}_f\left(a, \frac{a+b}{2}\right) - \mathcal{S}_f\left(\frac{a+b}{2}, b\right) \right|.$$

This a posteriori error analysis allows a *recursive* definition of the Simpson's that can drive the total error near a certain value. That is, for a set tolerance on the error, `tol`, if the next level of Simpson's rule satisfies the inequality

$$\left| \mathcal{S}_f(a, b) - \mathcal{S}_f\left(a, \frac{a+b}{2}\right) - \mathcal{S}_f\left(\frac{a+b}{2}, b\right) \right| < 15 \cdot \text{tol},$$

then the total error is estimated to be below the given error tolerance

$$\left| I(f) - \mathcal{S}_f\left(a, \frac{a+b}{2}\right) - \mathcal{S}_f\left(\frac{a+b}{2}, b\right) \right| < \text{tol}.$$

Otherwise we subdivide the intervals, apply Simpson's rule again and check against the tolerance once more.

This divide-and-conquer strategy is the general idea behind an *adaptive quadrature rule* that automatically refines the approximation to distribute the quadrature nodes in a nonuniform way. (Worry not, you don't have to implement this).

- (c) Download the script `adaptiveSimpson.py` that performs the adaptive Simpson's rule and run it for the current problem by setting the tolerance to be `tol = 1e-3`

```
from adaptiveSimpson import adaptiveSimpson
I_adapt, x_adapt = adaptiveSimpson(f, 0, 3, 1e-3)
```

What approximation does it give for the integral and how many sub-intervals does it use?

```
I_adapt =
```

```
len(x_adapt) =
```

What is the error between the adaptive quadrature result and the previously computed value `I_true`? Is it near the tolerance you set?

```
abs(I_true - I_adapt) =
```

- (d) Use the script `plot_composite_quad.py` to plot the new set of nonuniform quadrature nodes

```
plot_composite_quad(f, x_adapt)
```

How is this distribution of quadrature nodes different compared to your plot from Part (b)?

4.4 Numerical methods for initial value problems

Lastly, we will explore the accuracy and use of a computer to approximate the solution of an initial value problem governed by an ordinary differential equation. For this we employ two methods:

- 1) Forward Euler.
- 2) Four-stage Runge-Kutta.

For a general initial value problem

$$y' = f(t, y), \quad t \geq t_0, \quad y(t_0) = y_0.$$

We assume that there is a uniform discretization of the time variable from the initial point t_0 to the final time T with step size h

$$t_i = t_0 + ih, \quad i = 0, \dots, n, \quad h = \frac{T - t_0}{n}.$$

Then these two ODE time integrators take the form

$$\text{Forward Euler : } y_{i+1} = y_i + hf(t_i, y_i)$$

and

$$\begin{aligned} \text{Runge-Kutta : } y_{i+1} &= y_i + \frac{h}{6} (k_1 + 2k_2 + 2k_3 + k_4), & k_1 &= f(t_i, y_i) \\ & & k_2 &= f\left(t_i + \frac{h}{2}, y_i + \frac{h}{2}k_1\right) \\ & & k_3 &= f\left(t_i + \frac{h}{2}, y_i + \frac{h}{2}k_2\right) \\ & & k_4 &= f(t_i + h, y_i + hk_3) \end{aligned}$$

We see that both of these time integration technique are explicit where we only need to know solution information at the current time t_i in order to advance the solution to the next time t_{i+1} .

- (a) Download the forward Euler code for `forwardEuler.py` and the skeleton routine `RungeKutta4.py` for the Runge-Kutta method. Use the structure of the provided forward Euler routine and implement the four-stage Runge-Kutta method.
- (b) To verify your implementation of the four-stage Runge-Kutta method consider the initial value problem

$$y' = \frac{5}{t} - \frac{1}{t^2} - 5ty^2, \quad y(1) = 1.$$

This ODE has the exact solution that you can use for comparison

$$y(t) = \frac{1}{t}.$$

Integrate the solution from the initial time $t_0 = 1$ up to the final time $T = 25$. You can define the right-hand-side as an anonymous function of two variables

`f = lambda t, y: 5 / t - 1 / t**2 - 5 * t * y**2`

The exact value at the final time is $y(25) = 0.04$. Fill in the table below for decreasing values of the time step size h and compute the error ratio.

h	<code>abs(0.04 - y(end))</code>	Error Ratio	Does it match the theoretical prediction for the fourth order Runge-Kutta method?
0.2		—	
0.1			
0.05			
0.025			

- (c) Numerically investigate the claim that Runge-Kutta is more stable than forward Euler. Use the same initial value problem from Part (b) and, again, integrate up to the final time of $T = 25$. In the table below report the approximate value of the solution y at the final time, i.e. `y(end)` for both methods.

Tips: Plot the solution data to examine stability. Figures are easier to interpret than raw data.

h	Forward Euler	Four-state Runge-Kutta	For which values of the time step size h are the respective methods stable?
0.4			
0.25			
0.1			

- (d) The true power of a numerical method, such as the Runge-Kutta time integrator, is to solve a mathematical problem for which we **do not** know the answer. For example, the Runge-Kutta method can solve a system of ODEs which model a complex physical process, like chemical reactions or heat convection. To illustrate this power consider the Lorenz equations

$$\begin{aligned}\frac{dy_1}{dt} &= 10(y_2 - y_1), \\ \frac{dy_2}{dt} &= 28y_1 - y_2 - y_1y_3, \\ \frac{dy_3}{dt} &= y_1y_2 - \frac{8}{3}y_3,\end{aligned}$$

which are a system of nonlinear ODEs that exhibit chaotic solutions. That is, the solution is a point in three-dimensional space that evolves in time with incredibly strange, and often unpredictable behavior. In pop culture, this chaotic behavior is typically attributed to *the butterfly effect*.

Download the file `LorenzAttractor.py` from the course documents. This uses an implementation of the four-stage Runge-Kutta method to integrate the nonlinear ODE system above in time.

- (a) Scroll to bottom of the script and implement the right-hand-side function in **Lines 65–67** such that the script will solve the Lorenz equations.
- (b) After this, run the script up to a final time $T = 50$

`LorenzAttractor(50)`

This will play you a movie plotting the evolution of the point trajectory for this strange attractor.

- (c) There is no question for this final task, but it serves to demonstrate what we can do with some of the tools you have learned in this course!